A PEDAGOGICAL PAPER IN THE SERIES ON AUTONOMOUS SYSTEMS IN FINANCE

CLAUDE COWORK

A Worked Case Study Applying Reynoso's Disassembly Framework to a Documented Over-the-Counter Agentic Architecture

"The unit of approval is the capability graph, not the vendor name."

Alejandro Reynoso

JUDGE BUSINESS SCHOOL. UNIVERSITY OF CAMBRIDGE. JULY 2026

Abstract

This case study takes one publicly documented over-the-counter agentic platform, Claude Cowork, and maps its disclosed technical architecture directly onto the component vocabulary developed in the lecture reading: the agentic loop, the orchestrator-subagent pattern, procedural Skills, the Model Context Protocol connector chain, execution environments, and the knowledge-vault epistemic hierarchy. Each component is scored against what Cowork's own documentation confirms is present, partially present, or absent. The exercise culminates in two direct applications of the lecture's own diagnostic instruments: the six-tier risk classification and the seven-object audit bundle. The finding is that Cowork demonstrably behaves as a cognitive runtime in the sense the lecture defines, while remaining, on the same evidentiary basis, short of an institutional operating system — most sharply on the dimension of native auditability, where the product's own documentation confirms that most of the required audit objects are not produced by default.

Keywords: Claude Cowork; agentic architecture; Model Context Protocol; Skills; risk tiers; auditability; institutional operating systems; AI governance.

Learning objectives

- apply the lecture's component vocabulary to a real, documented product rather than an abstract architecture;
- distinguish which elements of Reynoso's framework are confirmed present, partially present, or disclosed absent in a specific OTC platform;
- apply the six-tier risk classification to a concrete set of product capabilities;
- score a real product against the seven-object audit bundle and interpret the result for governance purposes;
- formulate a board-level recommendation using the lecture's understand-govern-advance sequence.

1 Why this case study exists

The lecture reading opens the autonomous engine conceptually: a model, a harness, an orchestrator, subagents, Skills, tools, Model Context Protocol (MCP) connectors, execution environments, knowledge vaults, permissions, validation, and evidence. A conceptual disassembly is necessary but not sufficient. The natural follow-up question for an institutional audience is whether a real, currently deployed OTC product actually exhibits these components, and if so, in what state of maturity.

This case study answers that question for one product, Claude Cowork, using only what its vendor has publicly documented. The method mirrors the lecture's own instruction: begin not with the product's marketing surface, but with its disassembled parts.

Pedagogical premise

A conceptual framework is validated by its ability to classify a real system, not merely to describe an idealized one. Where the framework and the documented product disagree, the disagreement is itself the finding.

2 Cowork as an instance of the agentic loop

The lecture's loop — observe, interpret, plan, authorize, act, evaluate, replan or stop — is not an abstraction here. It is, in substance, how Cowork's own documentation describes its behavior: the system analyzes a request and builds a plan, decomposes the work into subtasks where needed, executes code and shell commands inside an isolated environment, and coordinates multiple workstreams in parallel before returning a result for review. The *authorize* step in the loop corresponds to Cowork's documented permission gates: each tool call is checked against the user's connected-folder and connector permissions before it executes, and an organization may require this check on every call rather than allowing a persistent blanket approval.

Working observation

Loop stage in Reynoso $\longrightarrow$ Cowork's documented mechanism

Observe / Interpret $\rightarrow$ request analysis and planning
Plan $\rightarrow$ subtask decomposition
Authorize $\rightarrow$ per-call permission gate against connected-folder and connector scope
Act $\rightarrow$ code and shell execution inside an isolated sandbox
Evaluate / Replan $\rightarrow$ parallel workstream coordination and revision before delivery

The audit implication follows directly from the lecture: the final deliverable is only the last state of the loop; a reviewer must also be able to inspect the intermediate observations, plan revisions, and tool calls. Section 8 shows this is precisely where Cowork's architecture is weakest.

3 Orchestrator and subagents: an embryonic pattern

Cowork does not expose a named multi-agent organizational chart of the kind the lecture illustrates (research, analytical, artifact, and validation subagents operating as distinguishable roles). What is documented is a single control loop that decomposes complex work into subtasks and runs several workstreams in parallel. Applying the lecture's own governance question — *is the validation function sufficiently independent from the production path?* — the honest answer, on current public documentation, is that no independent validation subagent is disclosed. Cowork is better classified as an **orchestrator-worker pattern in embryonic form** than as the fully gov-

erned multi-agent graph the lecture describes as the target architecture.

Institutional lesson

Where the vendor has not disclosed an independent validation function, the institution must not assume one exists. Absence of evidence for a control is, for governance purposes, evidence of absence.

4 Skills, mapped

The lecture's definition holds without modification: the agent decides *what* must be done; the Skill explains *how the institution expects it to be done*; the tool supplies the capability with which the action is performed. Cowork loads structured instruction packets — a description together with reference material and, where relevant, scripts — that are matched to the task before the agent acts, using the same underlying mechanism documented for Anthropic's coding product.

What Cowork does *not* yet provide, on public documentation, is the Skill registry the lecture specifies as a governance requirement: owner, version, approval date, permitted agents, data classification, required tools, tests, and retirement rule. Skills currently arrive bundled with the product or authored by individual users; no enterprise Skill-governance console is disclosed.

Reynoso element	Present in Cowork?	Governance gap
Agent (what)	Yes — the control loop	—
Skill (how)	Yes — loaded instruction packets	No disclosed registry
Tool (capability)	Yes — connectors and execution	Partially auditable (Sec. 8)

If an institution adopts Cowork, constructing this registry is the institution's own responsibility, not a capability the vendor currently supplies.

5 The MCP chain, concretely

This is the component of the lecture that maps most precisely onto documented product behavior. Reynoso's chain — agent → MCP client → MCP server → application API → external system — is exactly how Gmail, Calendar, and Drive access is documented to work inside Cowork: a connector is authorized under the user's own existing permissions, so no privilege escalation beyond what the user could already do is possible; the server holds a short-lived, narrowly scoped token; and the underlying application programming interface is called on the server side rather than from inside the execution sandbox.

The lecture's directory-versus-custom-connector distinction also maps directly onto Cowork's own vocabulary: Anthropic distinguishes *verified* connectors, reviewed for security, reliability, and compatibility, from *custom* connectors, explicitly described as services that have not been verified by the vendor and that are reached from Anthropic's cloud rather than from the user's own device.

Least-structure principle, applied

Prefer a verified connector over a custom MCP server, and prefer either over the browser-control fallback described in Section 6. The less structured the channel, the greater the need for human verification.

6 Execution zones and the least-structure principle

The lecture's ranked table of execution paths — structured tool call, generated orchestration code, shell commands, browser automation, direct computer use, ordered from most to least controllable — is the single most useful lens for understanding Cowork's documented architecture. Two zones with materially different properties are disclosed:

- A **sandbox or virtual machine** for code execution and shell commands, network-filtered through a mandatory proxy that reaches only pre-approved destinations, and destroyed at the end of the session (remote mode) or isolated by the host operating system's own hypervisor (local mode).
- An optional **browser-control extension** that drives the user's actual browser session, which by design operates *outside* the sandbox's network controls, using the user's real cookies and existing logins.

Mapped onto Reynoso's execution-path ranking, the sandbox sits close to the "structured tool call / generated orchestration code" end: validated and contained. The browser-control extension sits close to the "direct computer use" end: the vendor itself advises against its use for sensitive material and disallows it entirely for regulated healthcare clients. A student applying the lecture's framework should identify this feature as the point where reasoning authority and execution authority are least separated — the proposal and the real-world action sit closest together, with the least structure in between.

7 Knowledge vaults: what exists, and the epistemic gap

Applying the lecture's four-class taxonomy — primary source, derived material, agent inference, human-approved knowledge — Cowork's documented behavior looks quite different from a persistent institutional vault. There is no disclosed, cumulative knowledge store that Cowork enriches by default over time. What the documentation confirms:

Reynoso class	Cowork's documented equivalent
Primary source	Read live per task through connectors; not copied into a persistent store
Derived material	An optional, user-level memory feature; not labeled with lineage or confidence
Agent inference	Generated fresh each session, inside a sandbox destroyed at session end
Human-approved knowledge	**Not natively supported** — no disclosed approval workflow

Epistemic control, restated

Nothing in the product enforces the source / derived / inference / approved distinction that the lecture identifies as the minimum requirement for cumulative institutional memory. If an organization allows confident-sounding Cowork outputs to enter a shared knowledge base without a named human approver, it has created exactly the self-reinforcing error loop the lecture warns against.

8 Applying the risk-tier table to Cowork

Using the lecture's six-tier scale, Table 1 places Cowork's *documented* capabilities at each tier and specifies the minimum control this specific product requires at that tier.

Tier	Cowork capability at this tier	Minimum control, applied
1 — Read-only analytical	Reading connected files, Drive, or Gmail via verified connectors	Restrict to verified directory connectors; log connector grants
2 — Internal artifact creation	Writing or editing files in connected folders; running code in the sandbox	Restrict connected folders; require per-call approval
3 — External communication	Drafting via the Gmail connector; full send requires a custom MCP with write access	Named human approval before any external send is enabled
4 — Transactional	Not natively supported; would require a custom MCP server to a transactional system	Do not connect without dual authorization built into that external server itself
5 — Administrative	Browser-control extension operating under the user's real credentials	Disable by default; enable only under privileged-access management

Table 1: Cowork capabilities mapped to Reynoso's risk tiers

The table demonstrates the lecture's central governance claim directly: the *same* product legitimately sits at markedly different risk tiers depending on which capability is switched on. The unit of approval is the capability graph, not the vendor name.

9 The audit bundle: Cowork against the seven-object standard

This is the sharpest diagnostic the lecture provides, and the sharpest documented weakness in the product. Table 2 scores Cowork against each of the seven audit objects the lecture specifies as the minimum requirement for institutional auditability.

Audit object (Reynoso)	Present in Cowork today?
Objective record	Implicit in the session transcript; not separately structured
Agent graph	Not exposed — no disclosed subagent-level breakdown
Context manifest	Partially — tool and Skill invocations visible during the session; not retained centrally
Execution trace	Visible live during the session; **not captured in audit logs, the Compliance API, or data exports**
Data and method lineage	Not systematically retained after the session ends
Control record	Permission checks occur live but leave no persistent, exportable record

Audit object (Reynoso)	Present in Cowork today?
Artifact manifest	The output file exists; version, hash, and release-status metadata are not automatically attached

Finding

On the vendor's own documentation, five to six of the seven audit objects the lecture specifies are absent by default. A supplementary, separately configured monitoring feed can partially reconstruct the execution trace and control record — but only if an administrator deliberately enables it, which places this compensating control closer to the lecture's *reconstructed after an incident* anti-pattern than to its standard of native, by-design auditability.

This is the single finding a board memo on this product should lead with: it is not a matter of interpretation, but a direct, documented gap measured against the exact standard the lecture defines as the bar for institutional use.

10 The verdict, in Reynoso's terms

Applying the lecture's own diagnostic question — does the OTC product amount to an operating system? — to Cowork specifically:

Yes, as a cognitive runtime. It receives an objective, plans and revises, mediates access to files and connectors, executes code in a contained environment, and produces artifacts for review. Every item on the lecture's affirmative list is demonstrably present.

Not yet, as an institutional operating system. No disclosed corporate segregation-of-duties model exists beyond individual-user permission inheritance (Section 5); no unified knowledge-governance layer is disclosed (Section 7); no enterprise-wide Skill or capability registry is disclosed (Section 4); and, most decisively, no guaranteed audit bundle is produced by default (Section 9). Every item on the lecture's negative list is independently verifiable against Cowork's own documentation, not merely asserted by analogy to the framework.

Answer to the case-study question

Yes, the product behaves like an operating system for bounded cognitive work, exactly as the lecture predicts for a well-built OTC platform. No, it does not become the institution's operating system until the institution supplies the missing constitutional layer — and the clearest, most measurable absence is native auditability.

11 What this means for the board recommendation

Restating the lecture's own three-step sequence — understand, govern, advance — applied specifically to this product:

1. **Understand.** The components above are not hidden; they are documented, but distributed across separate technical articles rather than presented as a single architecture. Section 9 is the one finding that should not be diluted in translation to a non-technical audience.
2. **Govern.** Before any institutional rollout, build exactly what the product does not supply: a Skill registry (Section 4), a capability-graph approval process keyed to the risk tiers in Section 8, and a compensating audit-evidence pipeline configured before go-live rather than after an incident.

3. **Advance.** Once governed, the product is a reasonable platform for Tier 1–2 work under lockdown. Tier 3 work should require named human approval per the lecture's control principle. Tiers 4–5 should not be attempted using the product's native capabilities at all; attempting them would require the institution to build the missing segregation-of-duties and administrative-control layer entirely on its own infrastructure — at which point the institution is no longer using the product as an OTC runtime, but has begun constructing the governance architecture the lecture assigns as the institution's own responsibility.

Discussion questions

- The browser-control extension is the one documented component that operates outside its own sandbox. Using the execution-path ranking in Section 6, what governance gate would you impose before allowing this feature in a regulated business unit?
- The lecture asks whether Skills should be governed as documents, software, models, or a distinct asset class. Given that Cowork Skills can contain executable scripts, which classification would you defend, and does your answer change the vendor-risk assessment in Section 4?
- If an institution built the missing audit bundle (Section 9) as an internal compensating control, would that satisfy an external auditor — or does the absence of *native* auditability itself constitute a control deficiency, regardless of what is added afterward?

This case study applies the conceptual framework of Alejandro Reynoso, "Disassembling the Autonomous Enterprise Engine," Cambridge Judge Business School, July 2026, to publicly available product documentation for Claude Cowork current as of July 2026. All factual claims about Cowork's architecture are drawn from vendor-published documentation; none are inferred from the framework itself.